

ACTNverionMira: Omni-Language

Official Detailed User Manual v5.0

Welcome to ACTNverionMira. This language was designed to combine the most powerful features of Python, JavaScript, C, and HTML into a single, seamless 'Omni-Language'. It allows for backend logic and frontend UI rendering to exist in the exact same script.

1. Variables & Global State

Unlike strict languages, semicolons (;) are completely optional in ACTNverionMira.

Variables are dynamically typed and defined using the 'let' keyword:

```
let appName = "My Web App"
let version = 5
```

For global variables, use the 'Ustglobal' keyword:

```
Ustglobal isRunning = true
```

2. Arrays (Python-Style Lists)

You can define lists of data using square brackets. Access them via indices.

You can also natively query the size of an array using length().

```
let users = ["Alice", "Bob", "Charlie"]
let size = length(users)
print("First user: " + users[0])
```

3. Functions & Math

Functions are defined using the 'fn' keyword. They can accept arguments and return values.

```
fn calculateScore(points, multiplier) {
  let total = points * multiplier
  return total
}
let myScore = calculateScore(50, 2)
```

4. Native HTML & Web UI Rendering

One of the most powerful features of ACTNverionMira is native Web View integration.

You can construct HTML strings and use the 'render()' command to instantly display a real graphical user interface directly inside the IDE.

```
let buttonUI = "<button style='color:red;'>Click Me</button>"
render(buttonUI)
```

Concatenating logic with UI is simple:

```
let items = ["Apple", "Banana"]
let ui = "<h2>Items: " + length(items) + "</h2>"
render(ui)
```

5. Object-Oriented Programming (Classes)

ACTNverionMira supports full OOP. You can define classes, methods, and instantiate

them using the 'new' keyword.

```
class DatabaseConnection {  
    fn connect(url) {  
        return true  
    }  
}  
let db = new DatabaseConnection()  
print(db.connect("localhost"))
```

6. String Concatenation

You can dynamically build strings using the '+' operator.

This is particularly useful when building dynamic HTML interfaces or log messages.

```
let name = "Mira"  
let greeting = "Hello, " + name + "!"  
print(greeting) // Outputs: Hello, Mira!
```

7. Data Types Overview

ACTNverionMira is a dynamically typed language. The engine handles memory mapping.

- INTEGER: Whole numbers without decimals (e.g., 42).
- STRING: Text enclosed in double quotes (e.g., "Web").
- BOOLEAN: true or false keywords.
- ARRAY: Lists of any data type, accessible by 0-based index.

8. Console vs. UI Philosophy

The Omni-Language philosophy allows two forms of output:

- print(value): Outputs standard text to the Terminal Console.
- render(html): Parses the string as a web component and paints it onto the Web View.

This eliminates the need for separate frontend and backend frameworks.

9. Memory Scope Rules

- 'let' binds variables locally. Inside a function, they are destroyed upon return.
- 'Ustglobal' binds variables to the root Environment globally, preserving state.

Created natively by the ACTNverionMira Document Compiler Engine.